01 / DIGITUCK SYSTEM BLUEPRINT

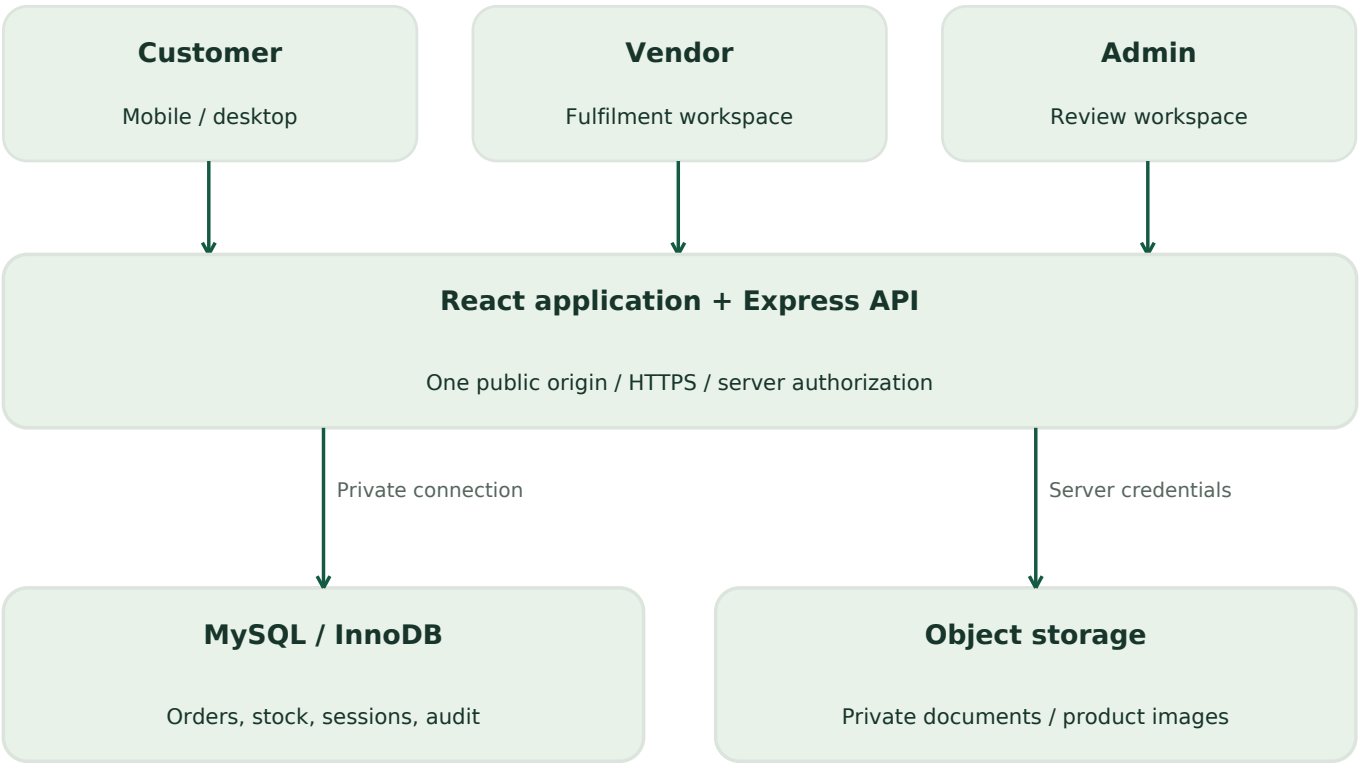
DigiTuck

System structure, interface direction and a practical Codex build plan.

Find local food. Order ahead. Collect when ready.

A focused first release for customers, local tuckshop owners and the people who review vendor applications.

3 roles	1 shop per order	Pickup only
Customer, vendor, admin	Clear ownership and simple stock handling	Payment arranged at collection



What this pack gives you

A resolved scope, three interface concepts, data and API design, safe order rules, acceptance tests, and copy-ready instructions for Codex. The full editable technical detail lives in the accompanying Markdown files.

Based on PHASE4-DigiTuck.pdf, 22 pages. These are proposed designs and implementation instructions; the application has not yet been built or tested. All interface data is illustrative.

Make the first release clear

The document contains conflicting ideas. These decisions keep the implementation coherent and traceable.

Source issue	Proposed resolution
Pickup-only scope on pp.3-4; delivery/payment diagrams on pp.8-9 and inputs on p.17	Follow the explicit MVP: no delivery, drivers or online payment.
Admin deferred on p.4 but required for verification on pp.2 and 19-22	Include a minimal admin approval and moderation console.
Responsive UI, validation, tests and history listed as future work	Treat them as baseline requirements for a usable first release.
React repeated in both presentation and application layers, p.5	Keep React UI; introduce an Express API for business logic.
Only four database entities on pp.10-11	Add shops, submissions, documents, sessions, events and stock records.
Different status names across pp.4, 10 and 19	Adopt one state machine; add rejected and proposed pending expiry.
Permanent deletion and separate admin table on p.15	Unified users with roles; deactivate accounts and preserve order history.
Client-side token removal suggested for logout	Use server-side session invalidation and protected cookies.

Explicitly deferred

Payments, delivery, multi-shop checkout, maps, native apps, marketing messages, loyalty, reviews, vendor staff accounts and advanced analytics.

Three roles, clear ownership

Use one authentication system. Enforce permissions on the server for every record, not just in navigation.

Role	Primary job	Boundary
Customer	Find shops, manage cart, place pickup orders, track and view history	Only own orders; cancellation before acceptance.
Vendor	Apply, manage products/stock, fulfil orders, inspect summaries	Only own shop; cannot self-approve or grant roles.
Admin	Review vendors, moderate products, suspend accounts, inspect audit	Dedicated audited actions; no vendor impersonation.
Guest	Browse approved shops and prepare a local cart	Must sign in as customer to place an order.

Navigation

Workspace	Menu
Customer	Discover / Orders / Account + contextual cart
Vendor	Overview / Orders / Products / Inventory / Reports / Business
Admin	Vendor reviews / Users / Products / Categories / Audit

Account rules

Proposed: one role per account and one shop per vendor. Public signup offers customer or vendor. Create the first admin with a trusted operator command. Keep the last active administrator protected from deactivation.

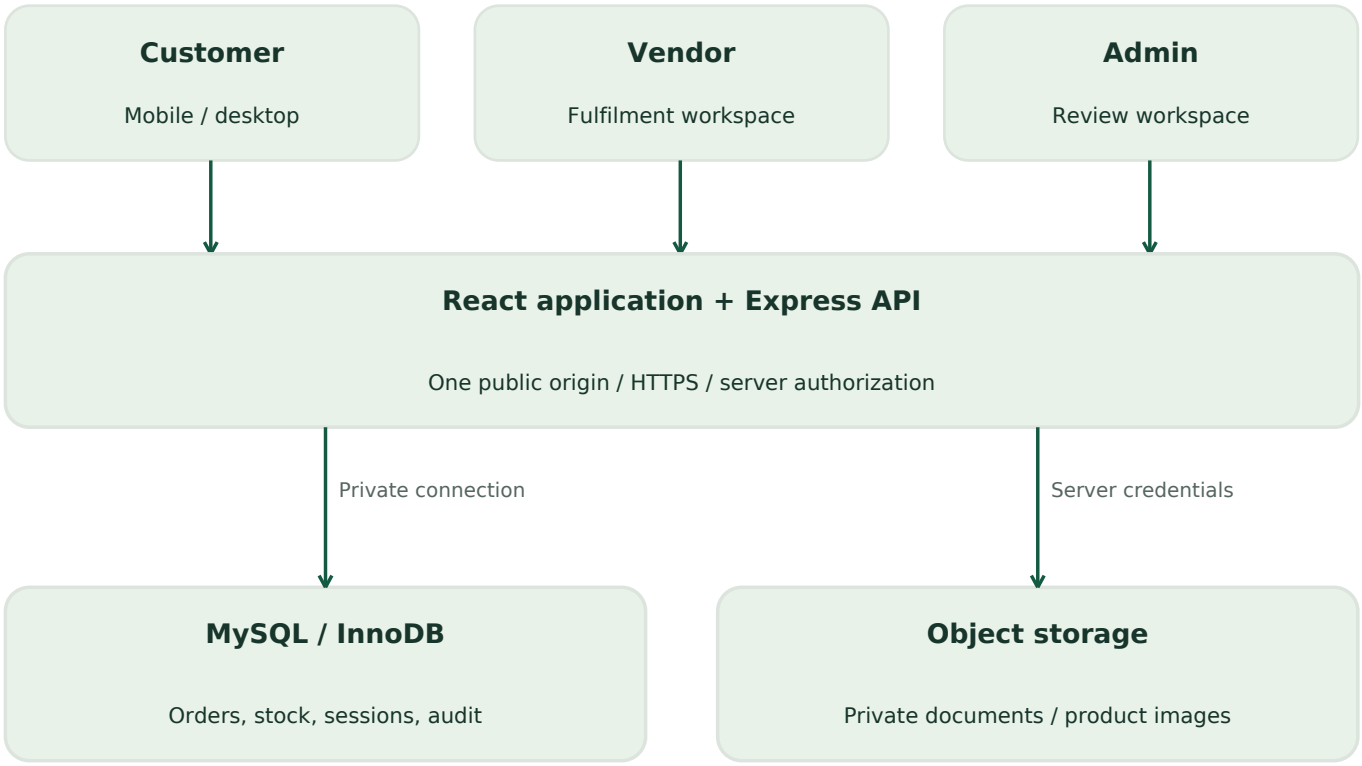
Pending vendors can prepare their catalogue and application. Only approved, active, open and unpaused shops can receive new orders. Suspended shops retain an explicit path for resolving existing orders.

Historical records remain usable

Archive products and deactivate users. Freeze the purchased product name/price and pickup address on each order.

Architecture that can grow

Keep the source choices of React, MySQL and Render. Add a small, explicit server layer.

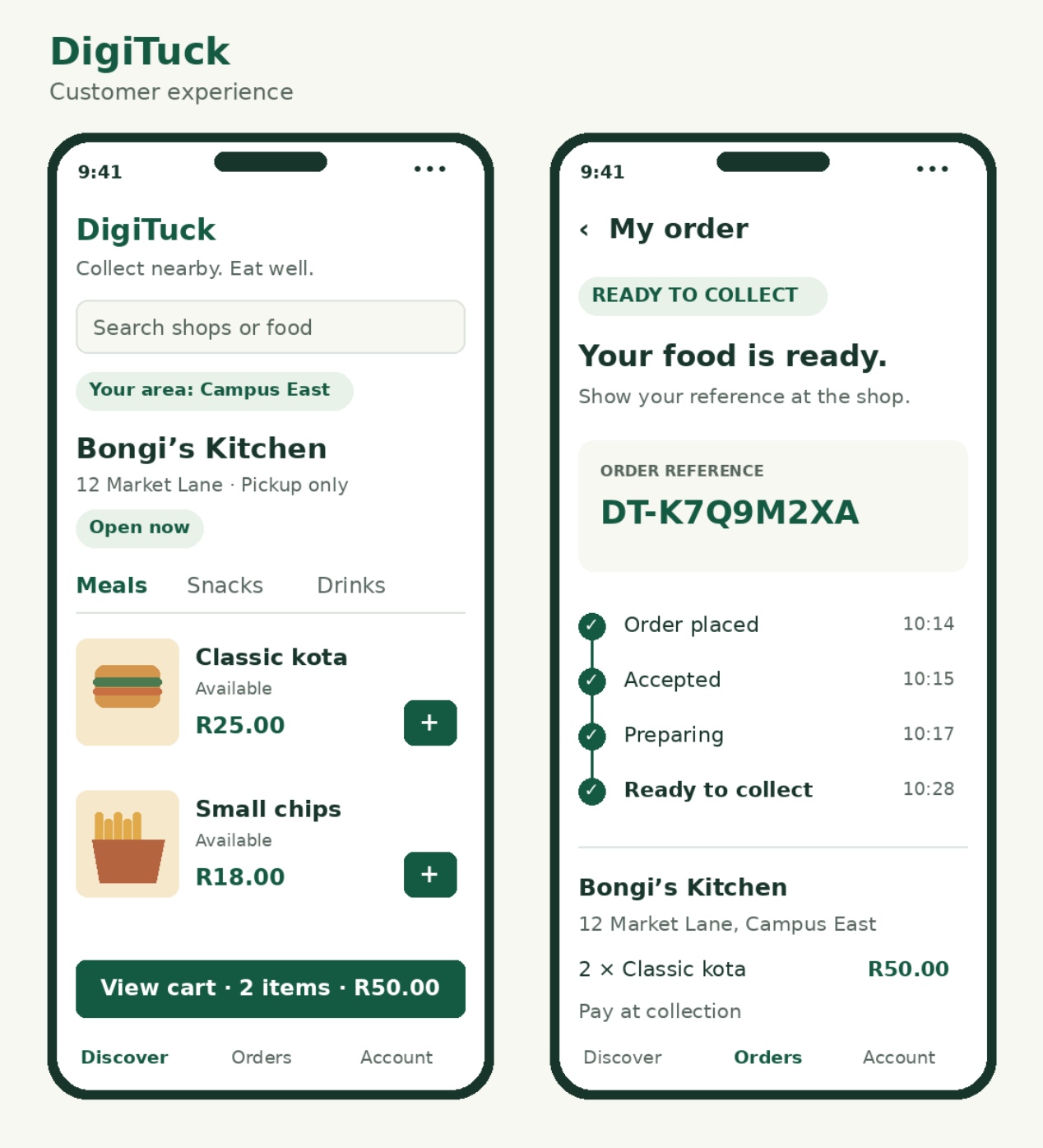


Layer	Proposed choice
Frontend	React, TypeScript, Vite, React Router, TanStack Query
API and validation	Express, TypeScript, Zod; service/repository separation
Data	MySQL 8.4 InnoDB; Knex + mysql2; versioned migrations
Identity	Server-side sessions in MySQL; secure cookie + CSRF
Testing	Vitest, Testing Library, MySQL integration tests, Playwright

One production origin serves the built React app and /api/v1. Use a Vite API proxy locally. Pin a supported Node LTS patch and compatible packages when implementing; commit the lockfile. [S4-S8, S10]

Customer: simple on a phone

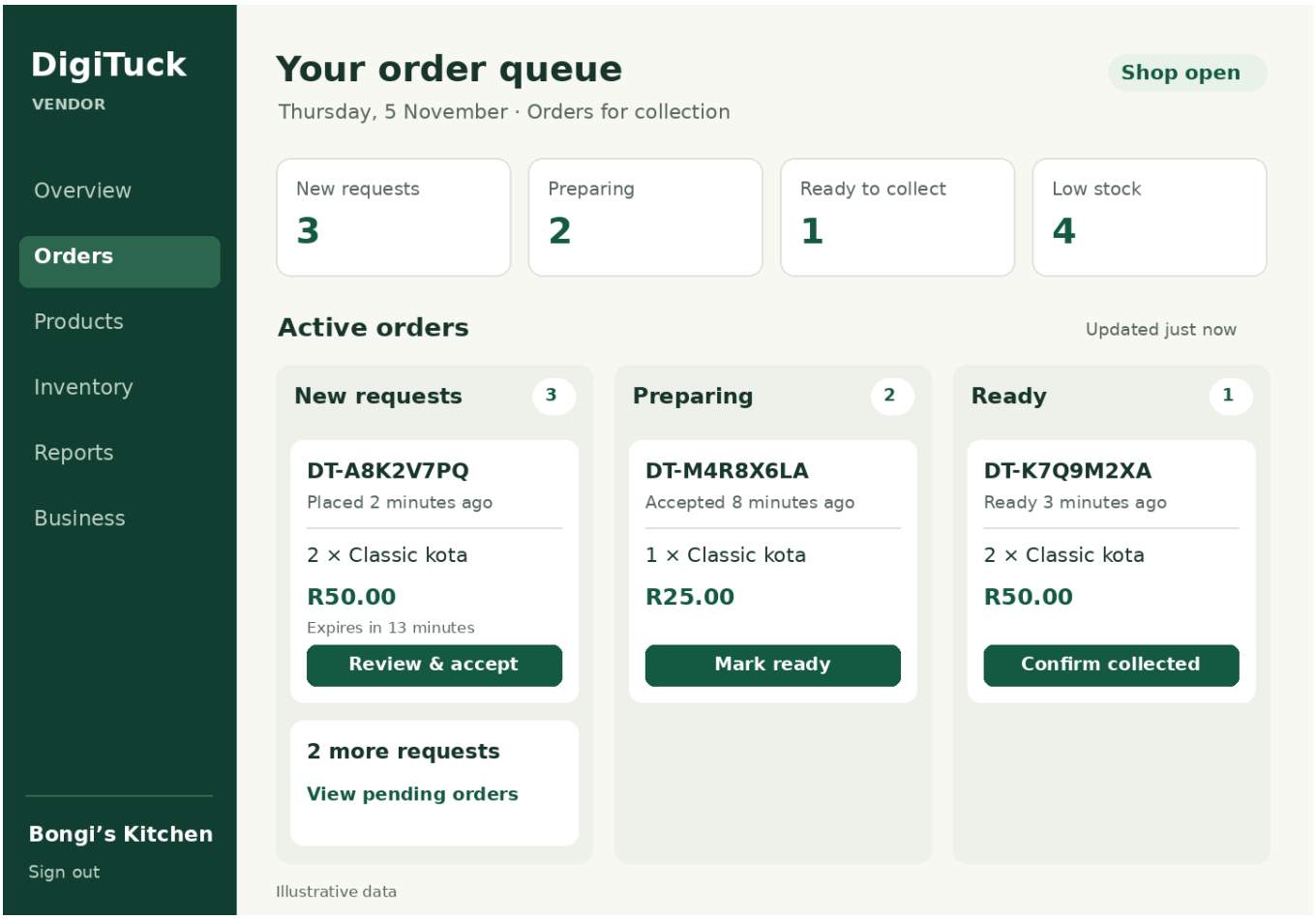
A catalogue that reaches food quickly, followed by a clear collection reference and honest status.



Concept screens with fictional data. Product artwork is illustrative. Customers pay at collection; the platform does not verify payment. No fabricated ETA or automatic offline order submission.

Vendor: focus on fulfilment

Show the next action, order age and items clearly. Keep products and stock one step away.

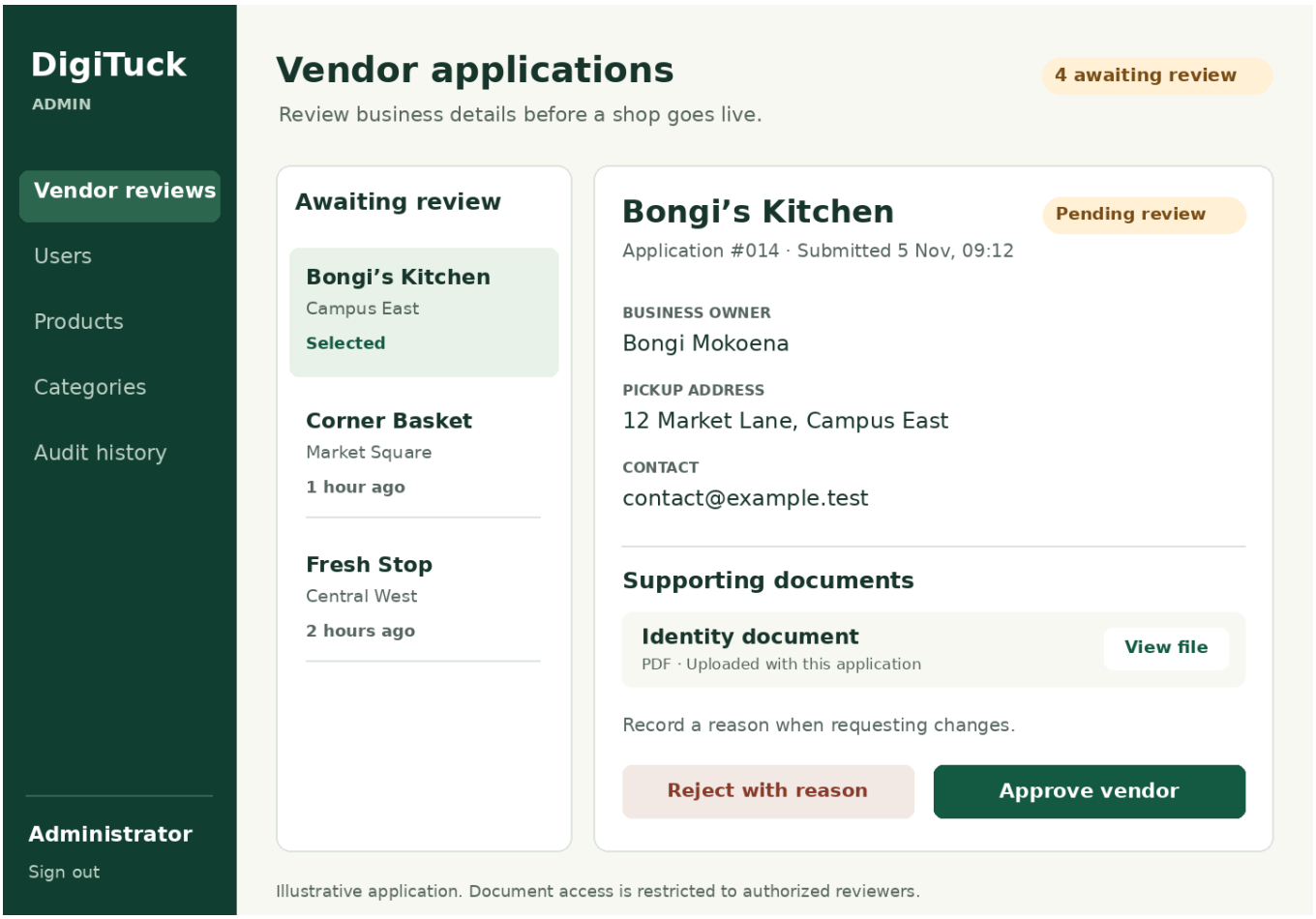


Area	Behaviour
Order cards	Reference, time, items, total and only permitted next actions.
New requests	Review before acceptance; pending deadline visible.
Stock adjustments	Change available quantity with a reason and version check.
Collection	Vendor confirms handover; do not subtract stock again.
Reports	Collected-order value and quantities, not verified payment revenue.

Mobile: replace desktop columns with status tabs and stacked order cards. Keep action buttons reachable and at least 44px high. Counts and names above are illustrative.

Admin: review before publishing

A focused application queue keeps review decisions separate from vendor editing.



Review the exact submission

Freeze business details and documents into an immutable application version. Approval must reference the version reviewed. If the vendor changes material identity, address or documents, require a new submission.

Keep documents private

Authorize every document request. Use controlled downloads or short-lived links; never expose identity documents in public shop pages. Reject decisions require useful reasons, and every approval, rejection or suspension is audited.

Document types are an operator decision. The source mentions identity/verification documents but does not establish a legal checklist. All application data in this concept is fictional.

A consistent design system

Warm, direct and economical with mobile data. Prioritize tasks, readable prices and clear feedback.

Design element	Rule
Colours	Green #145A43; cream #F7F8F2; white cards; text #17352B; orange #F0AD4E with dark text.
Type and spacing	System sans; 16px body; 24-32px headings; 4px spacing scale; 12px cards.
Responsive layout	320-767px mobile; 768-1023px tablet; 1024px+ desktop; max content width 1200px.
Accessibility	Labels, visible focus, keyboard support, 44px touch targets; never colour alone.
Reusable components	Shop/product cards, quantity stepper, cart summary, order timeline, status badge, form field, data table and document checklist.

States are part of the design

Situation	Customer-facing response
No connection	Cart stays on device; explain offline state; checkout disabled.
Different shop added	Ask before replacing the existing shop cart.
Price or stock changed	Show refreshed values; require review before new submission.
Order request timed out	Check using the same request key; do not create another order.
No results or failed load	Useful empty state or retry; never a blank screen.
Status updates	Refresh active visible orders; preserve content during refresh.

The complete route inventory and component rules are in 01_SYSTEM_BLUEPRINT.md. There are no visual-only controls for deferred features.

The records the system needs

Expand the four-table source model to support ownership, review, history and safe retries.

Entity	Main responsibility
users / sessions	Identity, roles, active state, secure session lifetime and revocation.
shops	Owner, pickup address, area, hours, approval state, pause and versions.
vendor_applications	Immutable submitted business profile and review result/version.
vendor_documents	Private object references tied to the submitted version.
categories / products	Catalogue, exact prices, visibility, moderation and available stock.
orders	Customer/shop ownership, state, total, reference, expiry and pickup snapshots.
order_items	Purchased name/price/quantity plus stock-return/write-off disposition.
order_events	Append-only status timeline, actor, reason and time.
inventory_movements	Signed reservation, release and adjustment records.
idempotency_requests	Unique request key, canonical content hash and committed result.
audit_events	Approval, moderation, stock and exceptional support history.
auth_rate_limits	Persistent expiring account/IP login-failure counters.

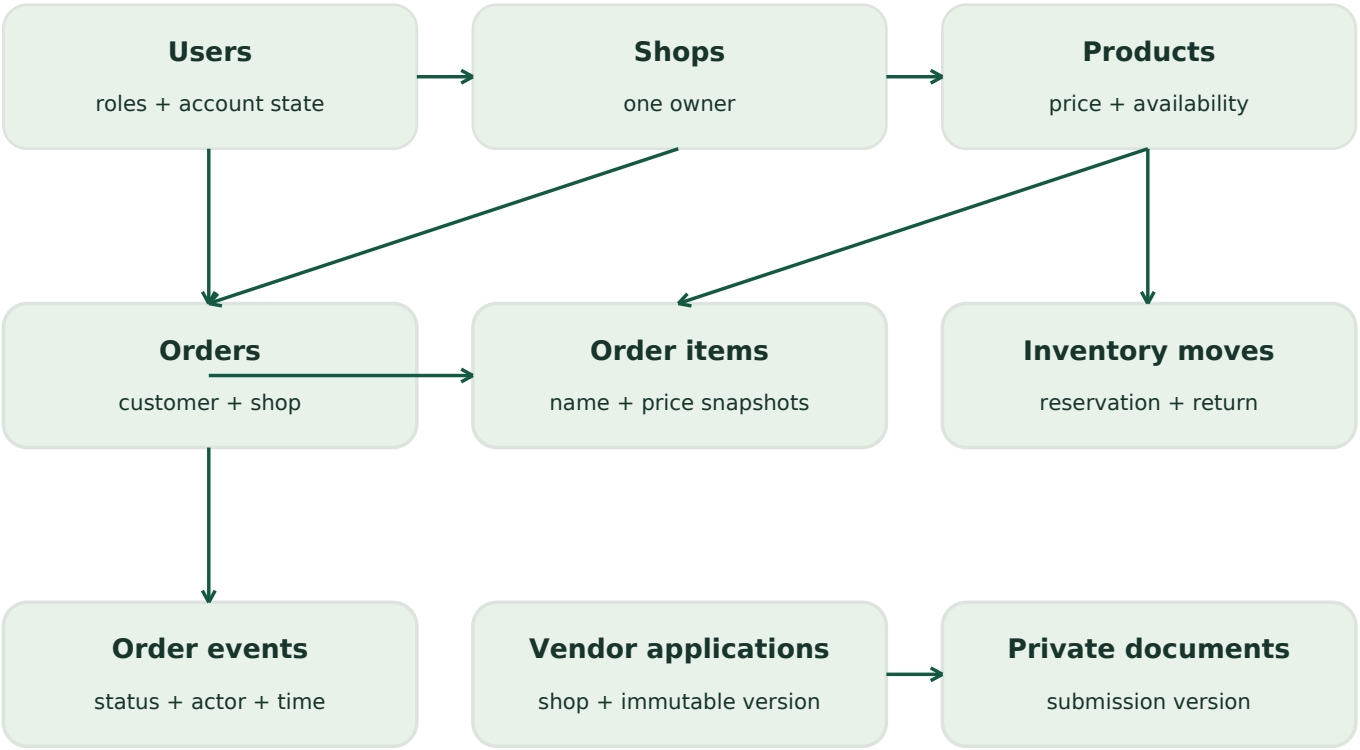
Deliberately absent

No payment, delivery, driver or server-cart tables in this MVP. Local cart persistence is enough until cross-device carts become a requirement.

Detailed fields, constraints and indexes are in the editable blueprint. Use UUID strings, utf8mb4, foreign keys, validation and database checks.

Relationships and invariants

The current catalogue can change. A confirmed order must remain historically correct.



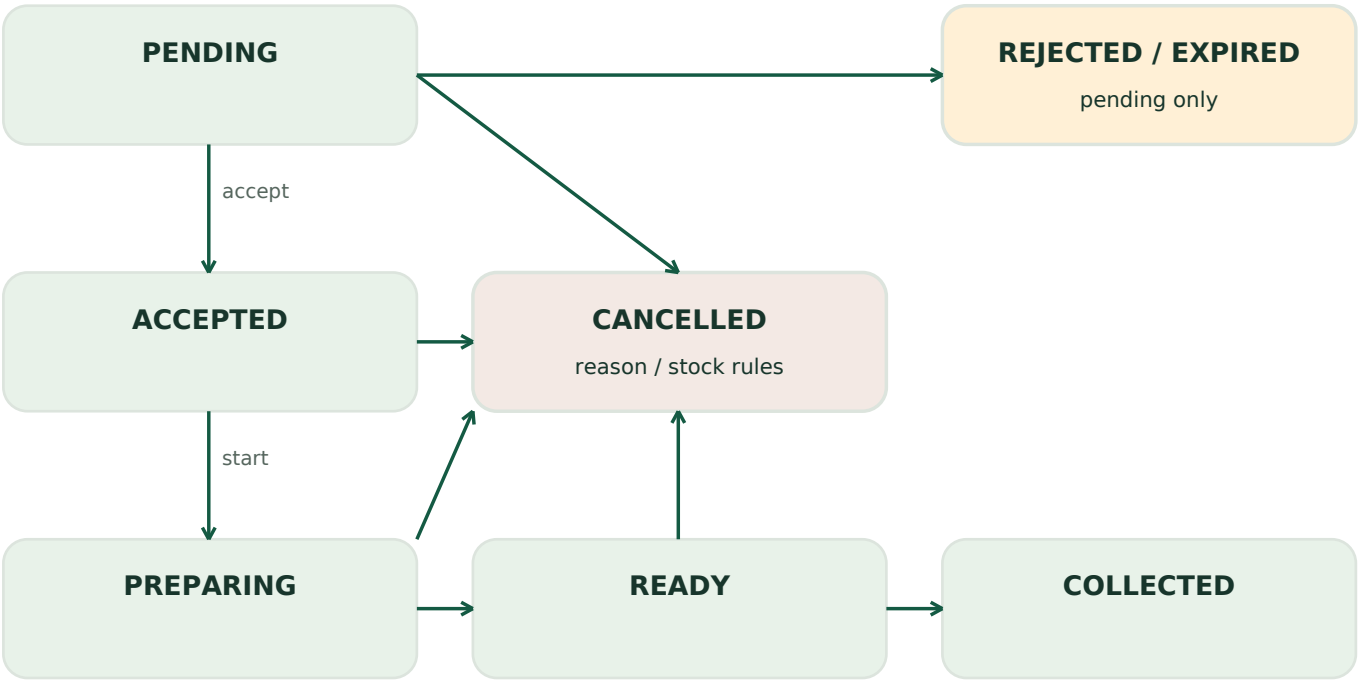
Focused relationship view. Authentication, idempotency and audit support tables are omitted here for clarity; the full entity inventory remains authoritative.

Rule	Implementation consequence
One vendor per order	All item products must belong to order.shop_id, checked on server.
Exact money	DECIMAL in MySQL; decimal strings in API; exact decimal/cents arithmetic.
Stable history	Snapshot product name/unit price, shop name and pickup address.
No negative stock	Lock and conditionally update availability inside the transaction.
No duplicate checkout	Unique user/operation/key and a request-content hash in MySQL.
No silent overwrite	Version checks for stock, product, status and review decisions.

Availability means stock still available to order. A reservation reduces it; collection does not reduce it again. Avoid treating this number as an independent physical inventory count.

One order state machine

Share this model between frontend labels, API validation and database records.



Action	Permission and inventory rule
Accept / prepare / ready / collect	Owner vendor follows the sequence; collection confirms handover.
Reject or expire pending	Return every reserved unit once. Proposed pending timeout: 15 minutes.
Cancel before preparation	Customer may cancel pending only; vendor/admin can cancel pending or accepted. Full return once.
Cancel after preparation	Vendor/admin gives reason and per-item resalable units. Return only those units; write off the remainder.
Terminal states	Collected, rejected, cancelled and expired cannot transition again.

Cancellation from PREPARING and READY requires explicit stock disposition. All exceptional admin actions are audited. The expiry duration is proposed, not specified by the source.

Checkout must be atomic

One transaction turns a reviewed cart into one order, one reservation and a repeatable result.

Step	Server behaviour
1. Validate identity/request	Customer session, CSRF, schema, quantity limits and request key. Reject writable role/status/total fields.
2. Claim idempotency key	Unique user + operation + key. Identical retry replays result; changed content returns 409.
3. Lock and recheck	Shop row, then products in sorted ID order. Check ownership, approval, hours, pause, prices and stock.
4. Build authoritative order	Exact totals; item/shop/address snapshots; random unique public reference.
5. Reserve and record	Conditional stock updates, movement ledger, status event and idempotency result.
6. Commit or roll back	All changes succeed together. Only confirmed success clears the customer cart.

Price changes need a fresh decision

A quote does not reserve stock. If current prices differ from the reviewed prices, return CART_CHANGED with refreshed values. Ask the customer to review and submit with a new key.

Protect against races

Use compatible lock order for checkout, suspension, stock edits, cancellation and expiry. Read IDs as needed, lock, then recheck. Bound deadlock retries and retain the request key. Test concurrency against real MySQL. [S6-S8]

Example: 10 available, reserve 3 = 7. Collection stays 7. Full cancellation restores 10 once. After preparation, restocking 1 leaves 8 and writes off 2.

API groups and contracts

Use /api/v1, consistent error envelopes and server-side ownership on every resource.

Group	Representative endpoints
Identity	GET /auth/csrf; POST /auth/register, /auth/login, /auth/logout; GET /auth/me
Account	PATCH /account; POST /account/password
Public catalogue	GET /shops, /shops/:slug, /shops/:slug/products, /products (cross-shop search), /categories
Checkout and customer orders	POST /checkout/quote, /orders; GET /orders, /orders/:id; POST /orders/:id/cancel
Vendor application	GET/PUT /vendor/shop; POST /vendor/documents, /vendor/shop/submit
Vendor catalogue	GET/POST /vendor/products; PATCH/DELETE /vendor/products/:id; POST image and stock-adjustment routes
Vendor fulfilment	GET /vendor/orders and /:id; POST /vendor/orders/:id/transitions; GET /vendor/reports
Admin review	GET /admin/vendors and /:id; POST /admin/vendors/:id/decision; GET private document download
Admin operations	Users/status, products/moderation, categories, exceptional order cancellation, audit
Health	GET /health/live and /health/ready

Contract details

Use UUID strings, UTC ISO timestamps, decimal money strings and bounded pagination. Return 409 for stale versions, invalid state, changed carts and idempotency conflicts; 422 for field/business validation. Auth/ownership failures use 401/403/404 appropriately.

The full endpoint catalogue, request example and error envelope are in 01_SYSTEM_BLUEPRINT.md. Codex must generate OpenAPI from the implemented contract and test direct HTTP access, not only browser navigation.

Security that supports the workflow

Focus on the risks already present: private documents, account access, stock changes and order ownership.

Boundary	Required behaviour
Passwords and login	Maintained hashing library; source-compatible bcrypt with measured work factor; reject overlong input rather than truncate. Generic errors and temporary throttles.
Sessions	Durable server store; secure HttpOnly SameSite cookie; rotate at login; idle/absolute expiry; real server-side logout/revocation.
Mutations	CSRF token plus Origin checks, explicit writable-field allowlists and role/ownership authorization.
Documents	Private random storage keys; validated signatures/types/sizes; authorized attachment or expiring URL.
Images	Decode/re-encode supported raster formats; strip metadata; reject SVG/HTML in upload endpoints.
Data and logs	Bound SQL; minimal DB privileges; redact passwords, tokens, signed URLs and unnecessary personal details.
Audit	Approval, suspension, moderation, stock changes and exceptional order decisions are attributable.

Proposed defaults

Password minimum 12 characters; bcrypt maximum 72 UTF-8 bytes; 30-minute idle / 8-hour absolute session lifetime; five failed logins per 15-minute window followed by a temporary throttle. Confirm operational policy before launch.

Production sessions must not use an in-memory store. SameSite helps but does not replace CSRF protection. Use maintained libraries and current official guidance. [S4, S9]

A realistic Render deployment

Keep MySQL and plan durable storage explicitly. Do not assume a complete production setup is free.

Component	Recommended deployment shape
Web application	One Render web service for Express and built React assets; one public origin.
Database	Private MySQL with a persistent disk, or an explicitly selected external managed MySQL provider.
Uploaded files	Durable object storage; private verification documents; server-controlled access.
Migrations	One controlled release step; separate staging data; avoid concurrent startup migrations.
Backups	Native MySQL backup to a separate durable destination; documented restore drill.
Operations	Readiness/liveness, redacted logs, secrets configuration, rollback instructions and support contact.

What the official hosting documentation changes

Render service filesystems are ephemeral unless data is on a persistent disk. Eligible paid services support disks; free web services do not. Free services can sleep when idle, which matters for active vendor order queues. [S2-S3]

For the documented self-hosted MySQL route, the disk mount is /var/lib/mysql. Keep database access private and use a database-native backup tool; raw disk snapshots are not an adequate restore plan for MySQL. [S1]

Cost and ownership decision

Choose hosting/database/storage providers and a budget before launch. The pack preserves MySQL; it does not silently replace it with PostgreSQL or provision paid resources.

Build in verifiable phases

Each stage should produce working behaviour and evidence, not disconnected mock screens.

Phase	Deliverable	Gate
0	Inspect repository and reconcile requirements	Decisions recorded; existing code preserved.
1	Foundation, schema, shared contracts, design tokens	Fresh install, migrate, seed, build.
2	Accounts, vendor application and admin review	Private docs; roles and approval enforced.
3	Catalogue, inventory and local single-shop cart	Real data; correct stock/price validation.
4	Transactional orders, states, expiry and returns	Concurrency, retries and rollback tested.
5	Complete dashboards, history, reports and UX states	Three roles pass end-to-end journeys.
6	CI, deployment setup and operations handoff	Repeatable staging; launch blockers explicit.

Suggested six-person ownership

Owner	Focus
A / B	Customer UI / vendor and admin UI
C / D	Identity and onboarding API / orders and inventory API
E / F	Database, contracts and integration / QA, CI and deployment

Pair database and QA owners with feature developers early. Review shared contracts and migrations, integrate small pull requests regularly, and track progress in docs/implementation-status.md. This is a sequencing plan, not an estimated delivery date.

Acceptance: prove the hard parts

The complete test matrix is editable in the pack. These checks are requirements, not claimed results.

Test group	Evidence required
AT01 / AT20 - reproducibility	Clean install/migrate/seed/build/start; SPA deep links and API errors behave correctly.
AT02 / AT14 - permissions	Direct API attempts across users/vendors, role injection and missing CSRF all fail.
AT03 / AT16 - verification	Correct review version; pending shop hidden; private documents inaccessible to others.
AT04 - final-item race	Two simultaneous customers; one order commits; stock never goes negative.
AT05 / AT07 - retries and failure	Double clicks/timeouts return one order; mid-transaction exception rolls everything back.
AT06 / AT11 - current versus historic	Price changes require review; existing order snapshots survive catalogue/address edits.
AT08 / AT09 / AT10 - transitions	Illegal/stale changes rejected; expiry/accept race safe; no duplicate restock; prepared-food disposition correct.
AT12 / AT13 - suspension and sessions	Suspension blocks checkout; deactivation/password change/logout revoke access.
AT15 / AT17 - usability	Offline recovery and keyboard flow; 360px/1280px layout review with no overflow.
AT18 / AT19 - reporting and recovery	Collected-only totals with correct timezone bounds; documented isolated backup restore.

Use the real database for concurrency

Mock tests and SQLite cannot demonstrate MySQL row-lock and transaction behaviour. Run these tests against a disposable MySQL instance.

Put the pack to work

The proposed defaults let implementation begin; a few operating choices remain for the owner.

Use with Codex

1. Open the project repository and attach the source specification plus this pack.
2. Copy the section marked PROMPT START to PROMPT END from 02_CODEX_MASTER_PROMPT.md.
3. Let Codex inspect the project, then implement and test each phase.
4. Use 03_PHASE_PROMPTS.md to resume or focus the next stage.

Decision	Proposed starting point
Launch geography / currency / language	Area-based search; ZAR; English; Africa/Johannesburg.
Shop ownership	One shop per vendor, one role per account.
Pending orders	Expire after 15 minutes; show deadline; vendor resolves accepted no-shows.
Cancellation	Customer before acceptance; vendor/admin afterward with safe stock disposition.
Documents and retention	Operator to define actual required documents, access and retention.
Hosting and recovery	Choose budget/providers, support/account recovery and backup recovery targets.

What is ready now

The system structure, screen concepts, full editable specification, requirement matrix and build prompts are ready. Application code, deployment and application test execution are the next project stage.

The original specification is preserved. Recommendations and scope resolutions are identified so they can be reviewed without being mistaken for confirmed requirements.

Sources and companion files

Requirements come from the supplied document. Technology details were checked against official documentation on 5 November 2026.

Primary source

PHASE4-DigiTuck.pdf, 22 pages. Key scope: pp.3-4; architecture: pp.5-9; schema: pp.10-11; workflows: pp.12-15; interfaces: pp.16-19; security and deployment: pp.20-22.

Official technical references

S1 - Render MySQL deployment
https://render.com/docs/deploy-mysql

S2 - Render persistent disks
https://render.com/docs/disks

S3 - Render free services
https://render.com/docs/free

S4 - Express production security
https://expressjs.com/advanced/best-practice-security/

S5 - Vite guide and runtime requirements
https://vite.dev/guide/

S6 - Knex transactions
https://knexjs.org/guide/transactions.html

S7 - Knex query builder
https://knexjs.org/guide/query-builder.html

S8 - MySQL 8.4 locking reads
https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

S9 - OWASP session management
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

S10 - Node.js release schedule
https://github.com/nodejs/Release

Companion files

01_SYSTEM_BLUEPRINT.md - detailed data model, rules and endpoints.
02_CODEX_MASTER_PROMPT.md - copy-ready implementation instruction.
03_PHASE_PROMPTS.md - staged tasks, resume and review.
04_REQUIREMENTS_MATRIX.csv - traceability.
design/ - screen concepts and editable Mermaid diagrams.